

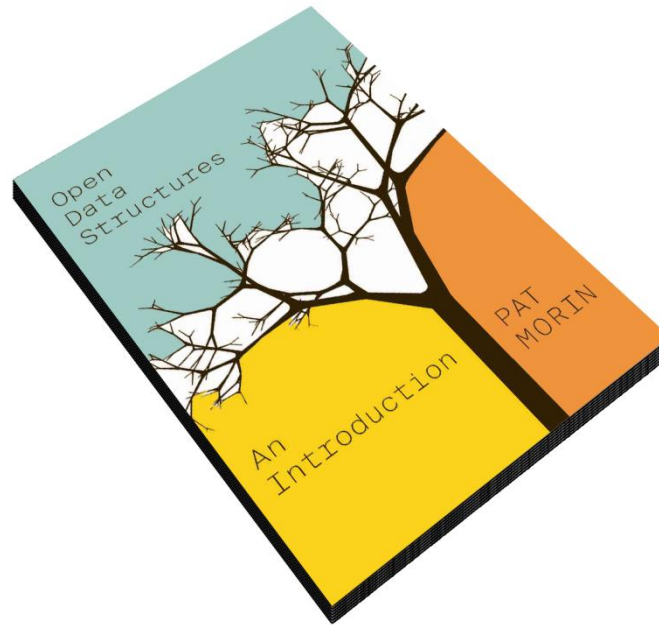
RootishArrayStack

COMP 2402

Abstract Data Types & Algorithms

Array-Based Lists

- Chapter 2



Wasted Space?

How much space is wasted (at any time) in these data structures?

- ArrayStack
- ArrayQueue
- ArrayDeque
- DualArrayDeque

Wasted Space?

How much space is wasted (at any time) in these data structures?

What does wasted space mean?

Any memory used in the data structure that does not store actual data (primitives) or a reference to the actual data (other objects) is considered wasted space.

Wasted Space?

How much space is wasted (at any time) in these data structures?

What does wasted space mean?

Any memory used in the data structure that does not store actual data (primitives) or a reference to the actual data (other objects) is considered wasted space.

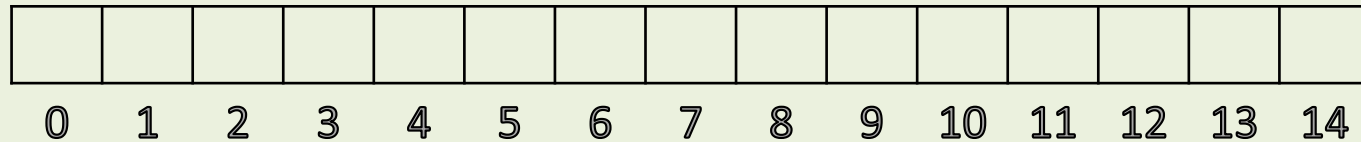
A data structure that stores n values requires $\Omega(n)$ memory just to store the data itself. We'll see later that any data structure that lets you add n values for storage requires $\Omega(\sqrt{n})$ additional (wasted) memory.

Wasted Space?

How much space is wasted (at any time) in these data structures?

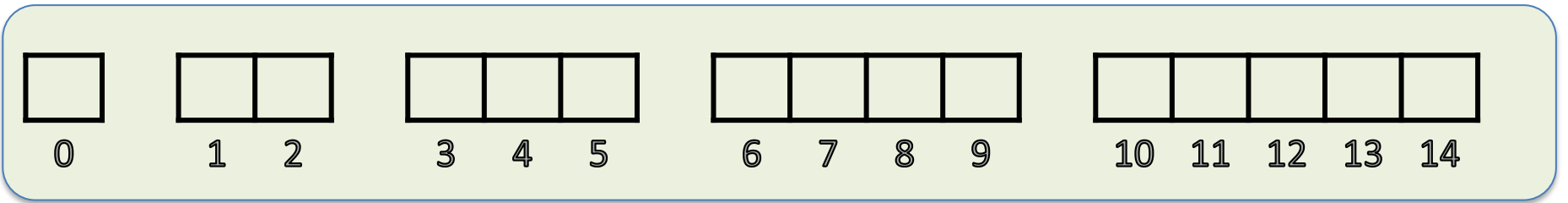
- ArrayStack
- ArrayQueue
- ArrayDeque
- DualArrayDeque

RootishArrayStack



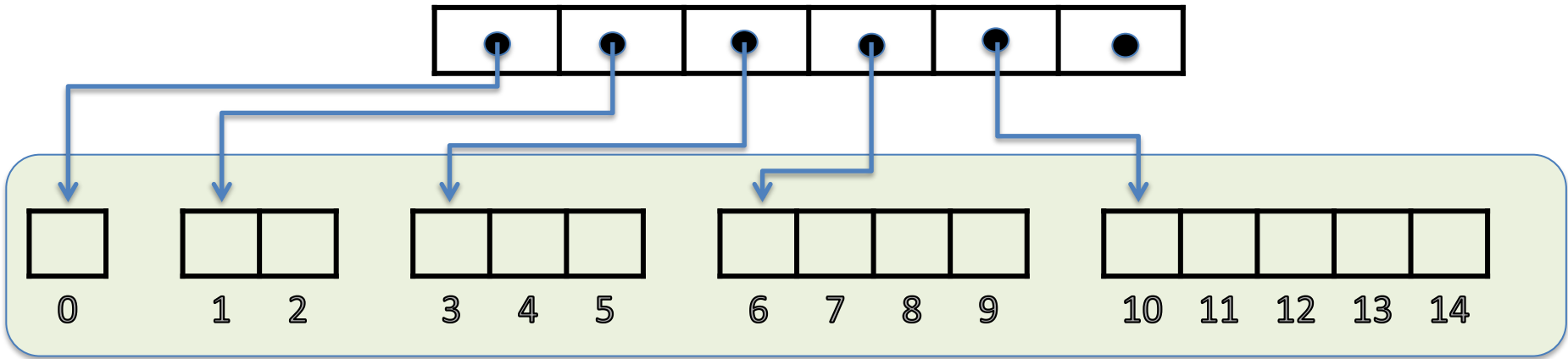
Imagine the ArrayStack with a backing array that is broken up into a sequence of arrays (with increasing sizes)

RootishArrayStack



Imagine the ArrayStack with a backing array that is broken up into a sequence of arrays (with increasing sizes)

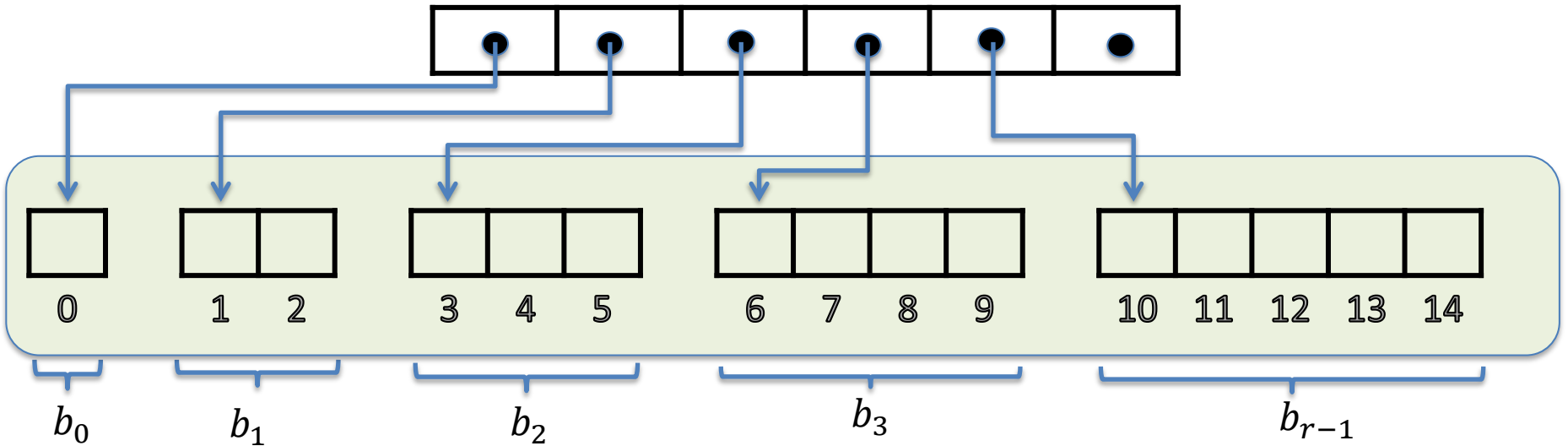
RootishArrayStack



Imagine the ArrayStack with a backing array that is broken up into a sequence of arrays (with increasing sizes)

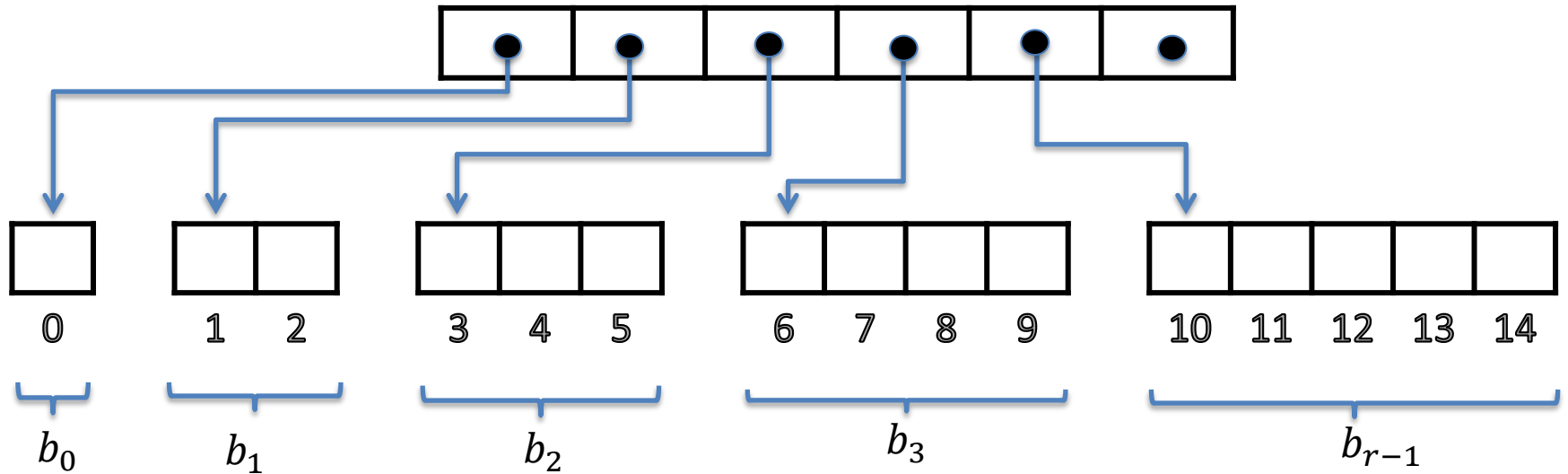
We need an array of arrays to access them!

RootishArrayStack



We can label each array (which will be called a **block**) by the numbers $0, 1, \dots, r - 1$ if there are r blocks.

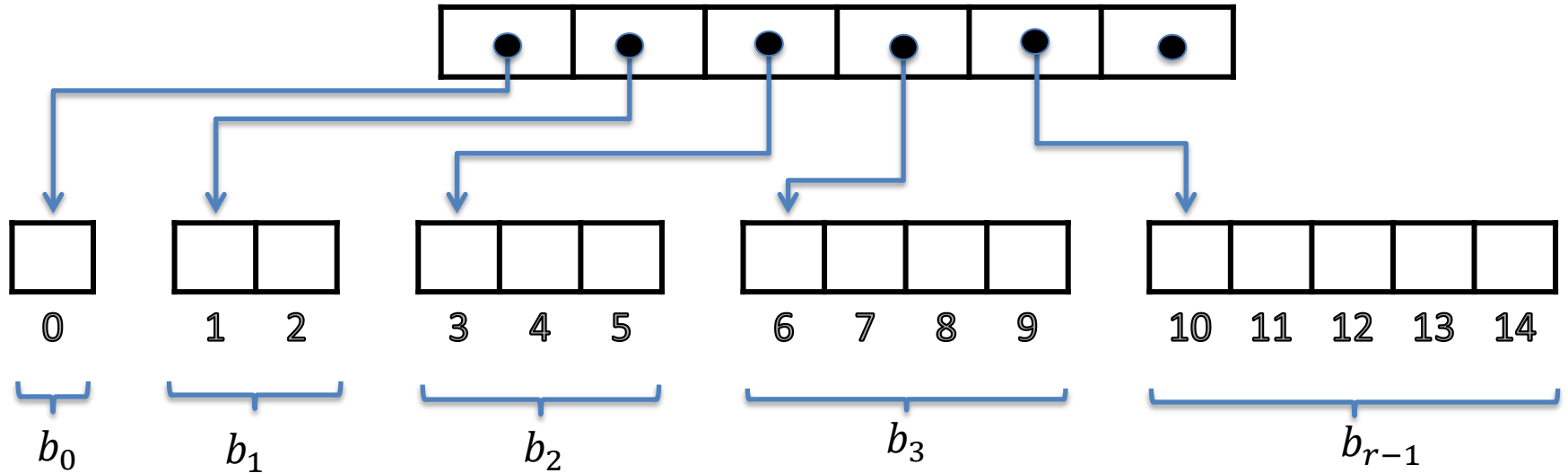
RootishArrayStack



Have r blocks of data (labelled $0, 1, \dots, r-1$)

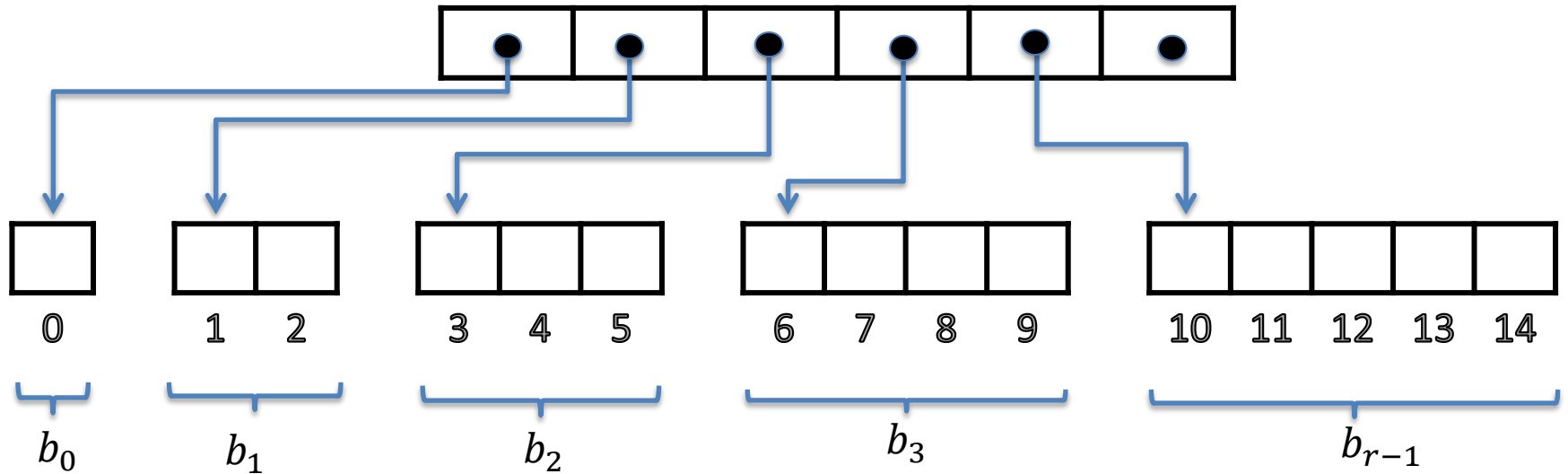
Block k (b_k) has size $k + 1$

RootishArrayStack



What is total amount of elements r blocks can store?

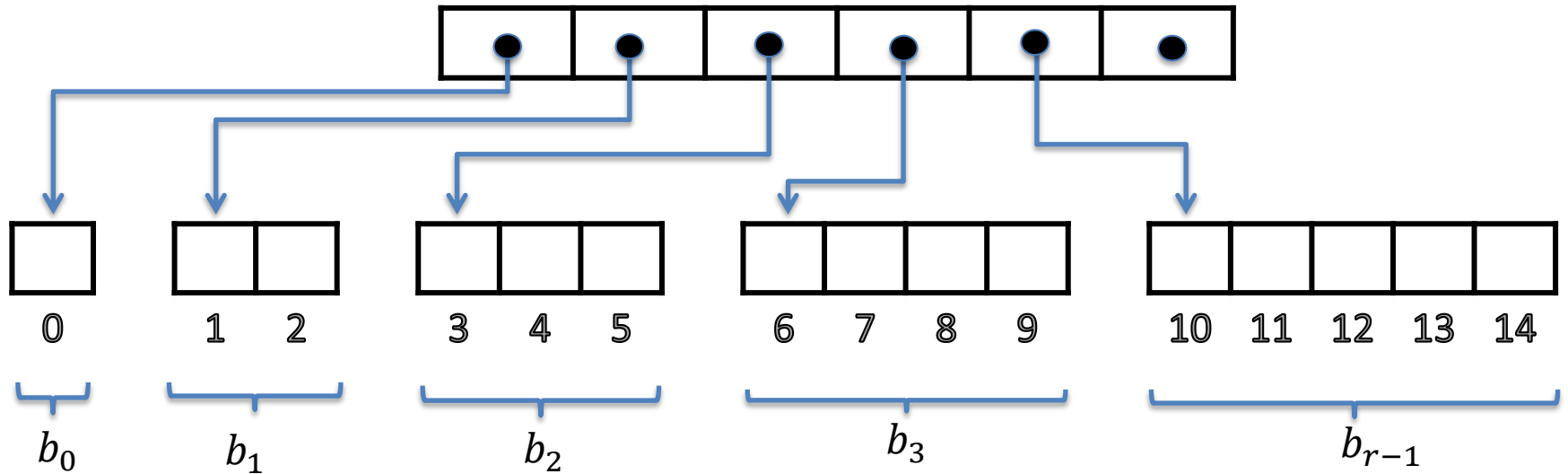
RootishArrayStack



What is total amount of elements r blocks can store?

$$\sum_{k=0}^{r-1} |b_k| = \sum_{k=0}^{r-1} k + 1 = \sum_{j=1}^r j = \frac{1}{2}r(r+1)$$

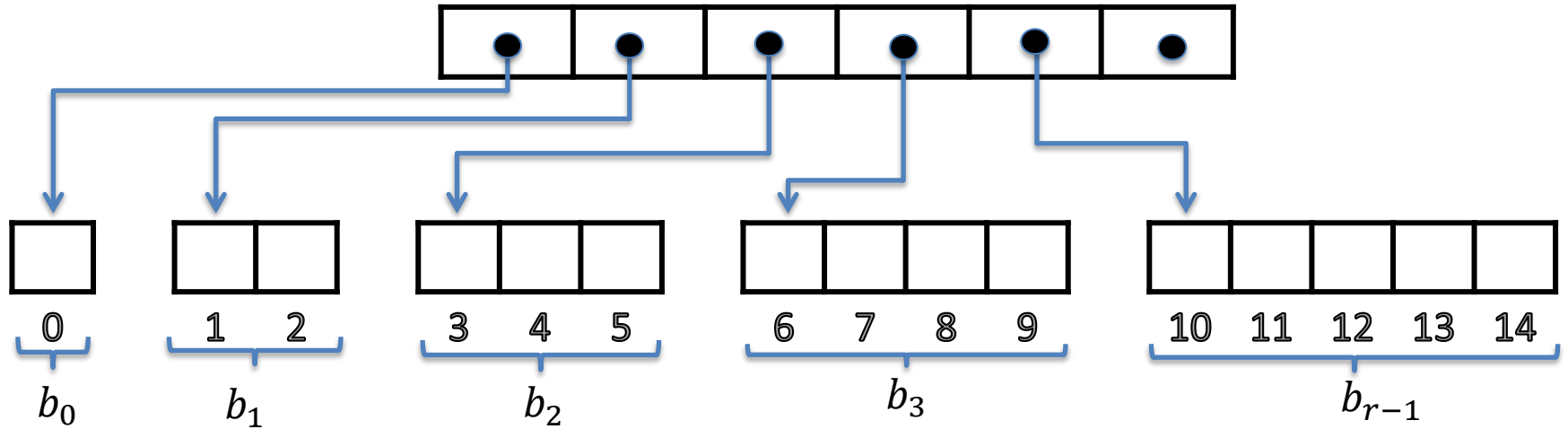
RootishArrayStack



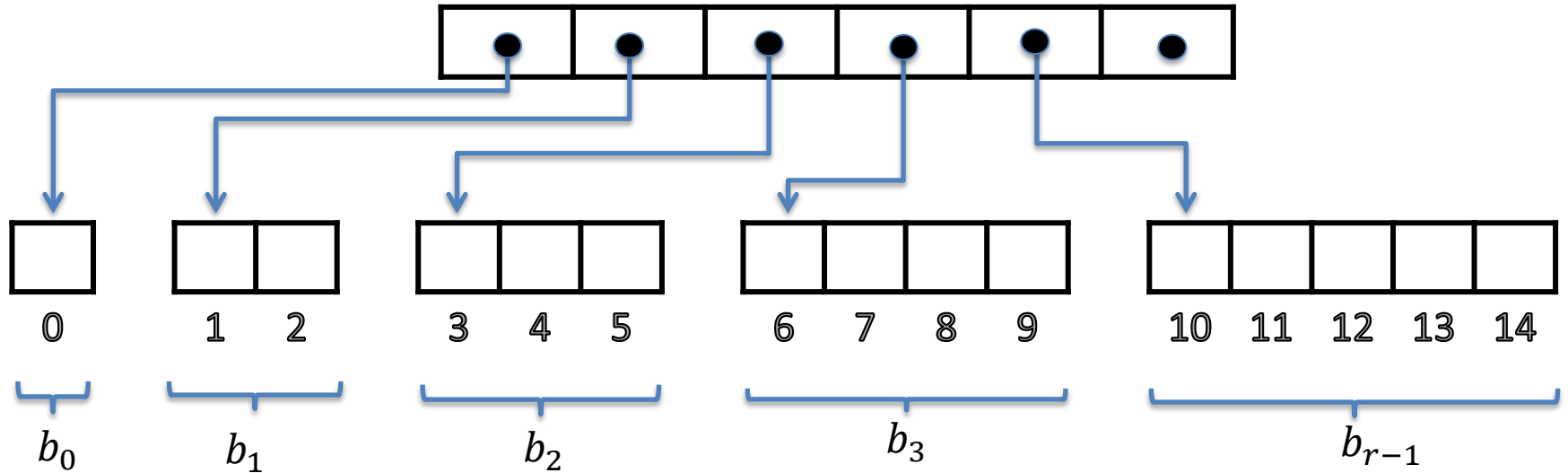
Which block is list index i located in?

Example: list[5] is found in block 2 (b_2) which is the 3rd block

RootishArrayStack



RootishArrayStack

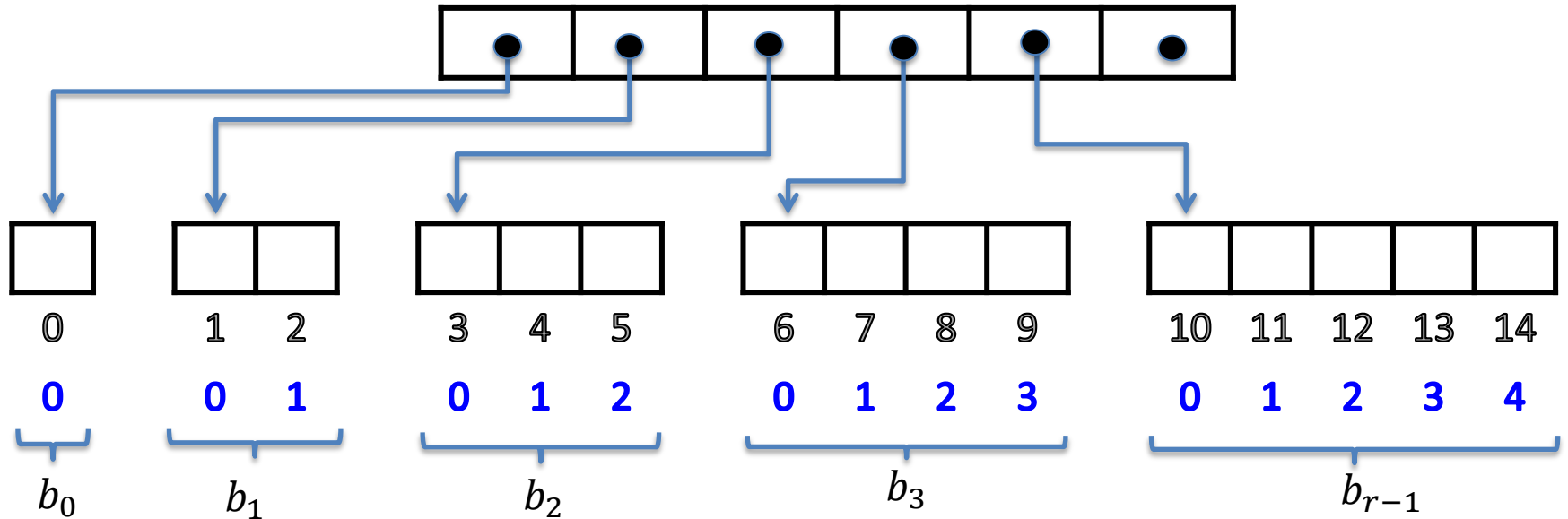


$$\frac{1}{2}(b+1)(b+2) \geq i+1$$

$$b^2 + 3b - 2i \geq 0$$

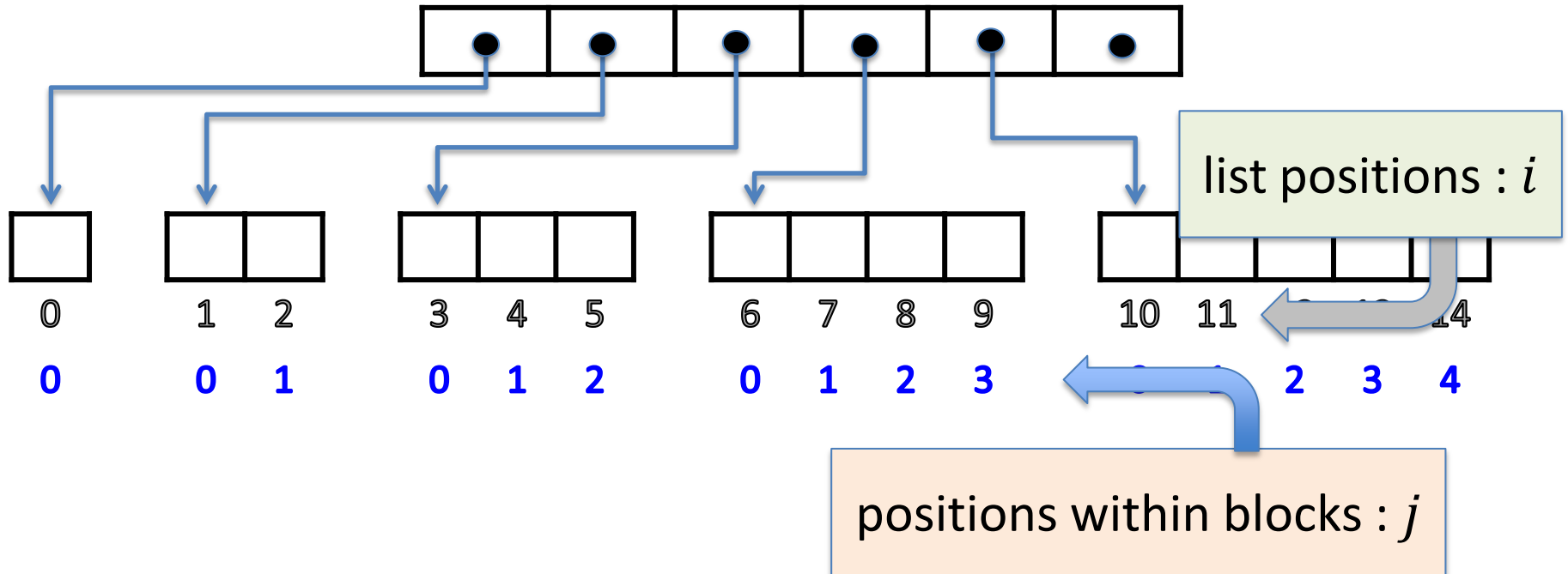
$$b = \left\lceil \frac{1}{2}(-3 + \sqrt{9 + 8i}) \right\rceil = \text{ceil} \left(\frac{1}{2}(-3 + \sqrt{9 + 8i}) \right)$$

RootishArrayStack



Where is list index i located within that block?
(what is the index within the block?)

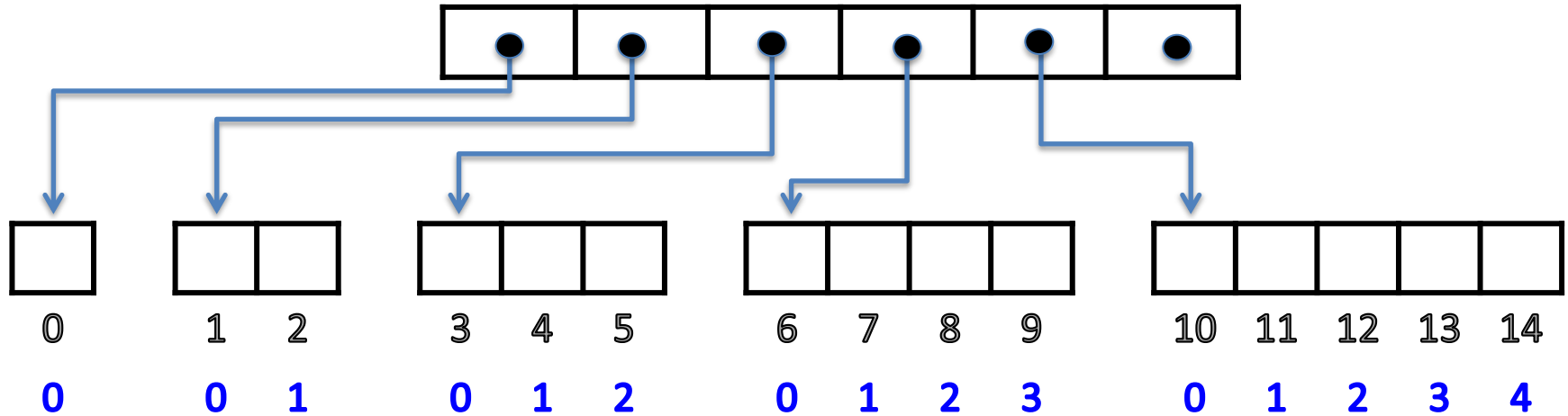
RootishArrayStack



Where is list index i located within that block?
That is, what is the index j within the block?

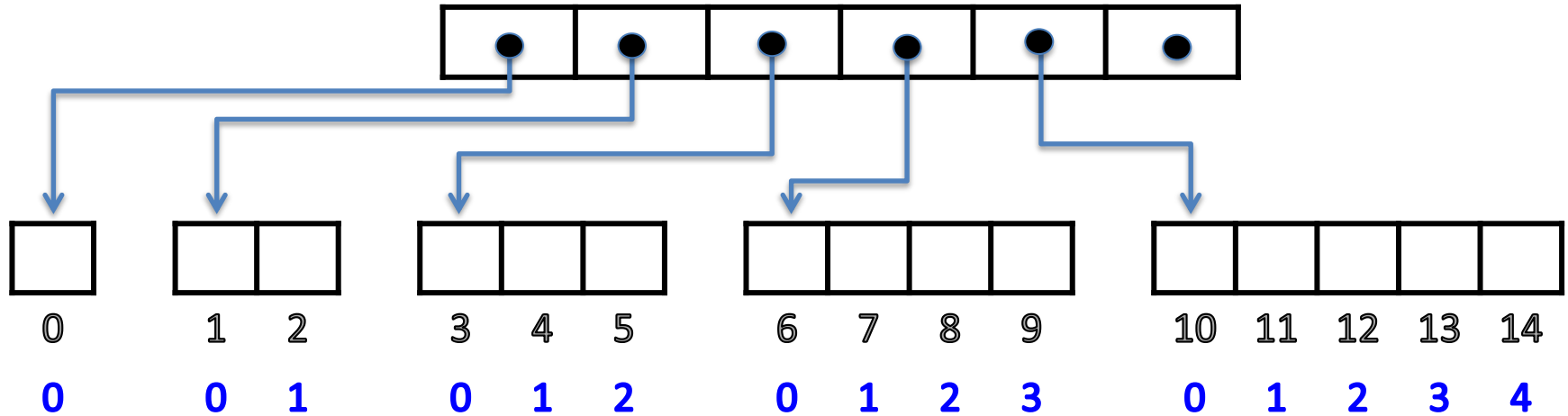
Example: list index $i = 7$ is located in position $j = 1$ of block 3 (b_3)

RootishArrayStack



Find j given i and b :

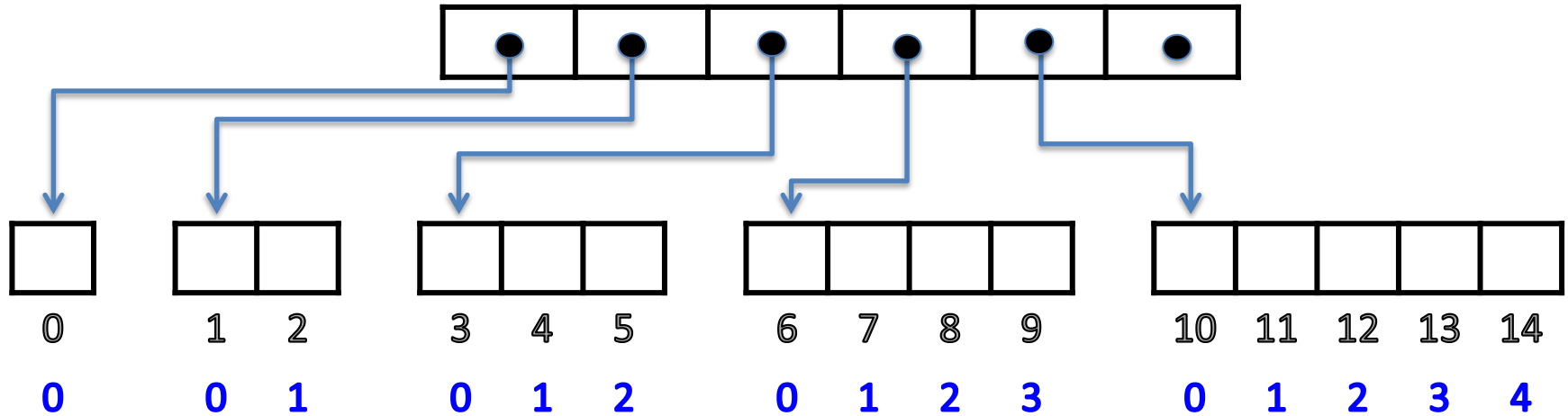
RootishArrayStack



Find j given i and b :

$$j = i - \frac{1}{2}b(b + 1)$$

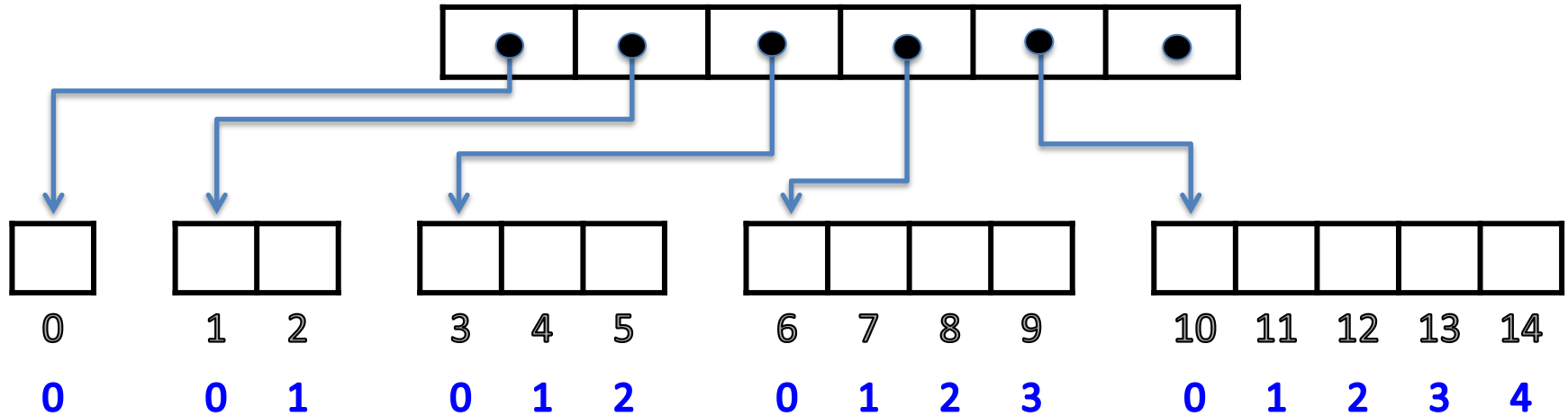
RootishArrayStack



What is cost of `get(i)` and `set(i,x)`?

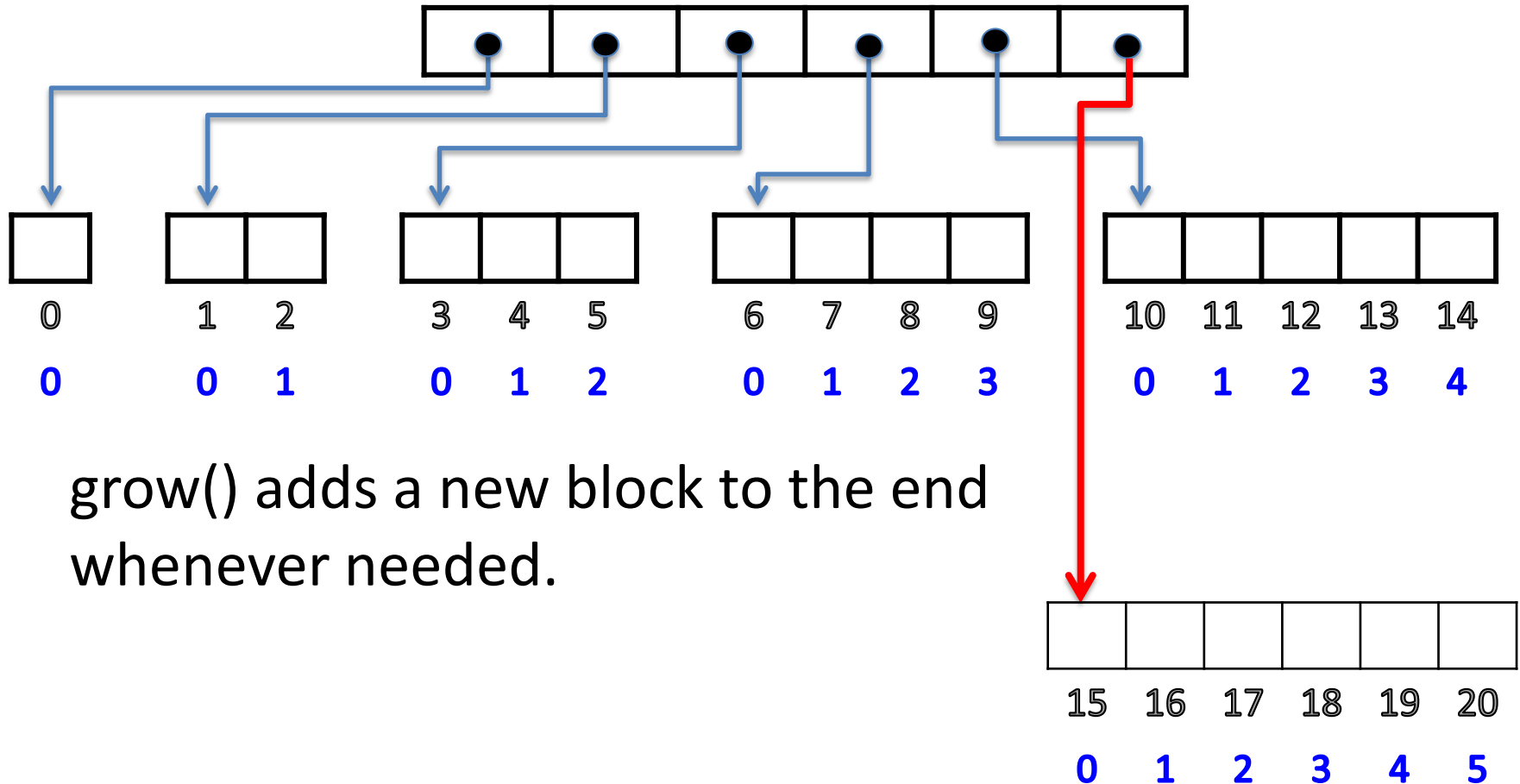
What is cost of `add(i,x)` and `remove(i)`?

RootishArrayStack

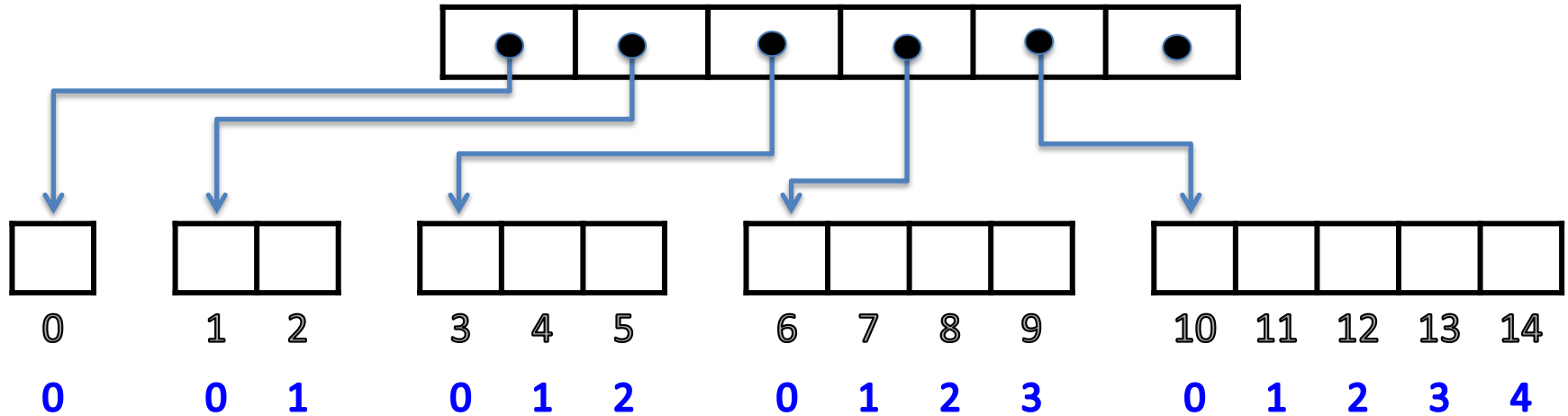


grow() adds a new block to the end
whenever needed.

RootishArrayStack

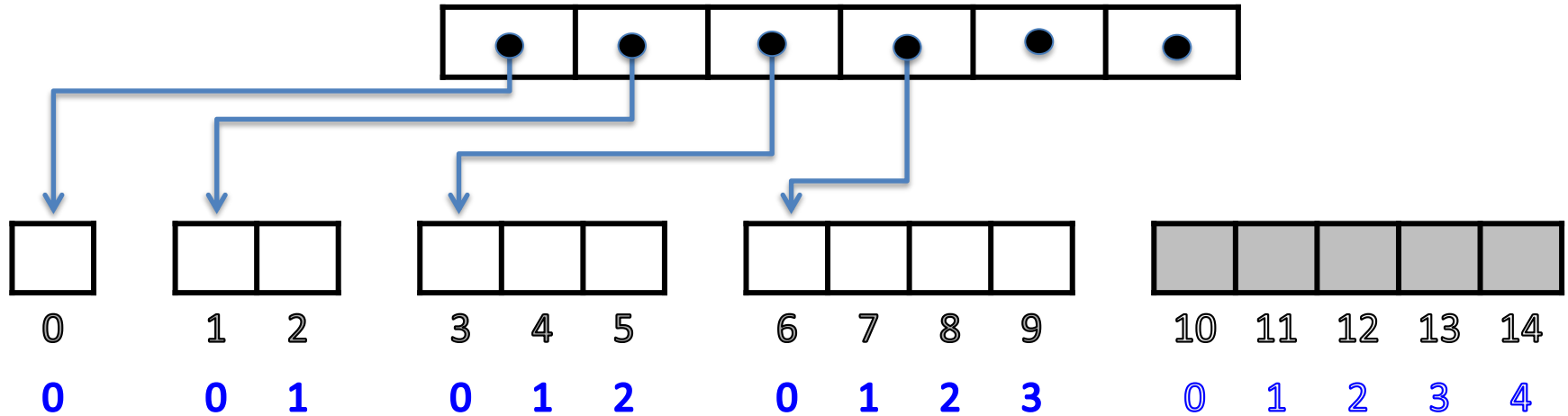


RootishArrayStack



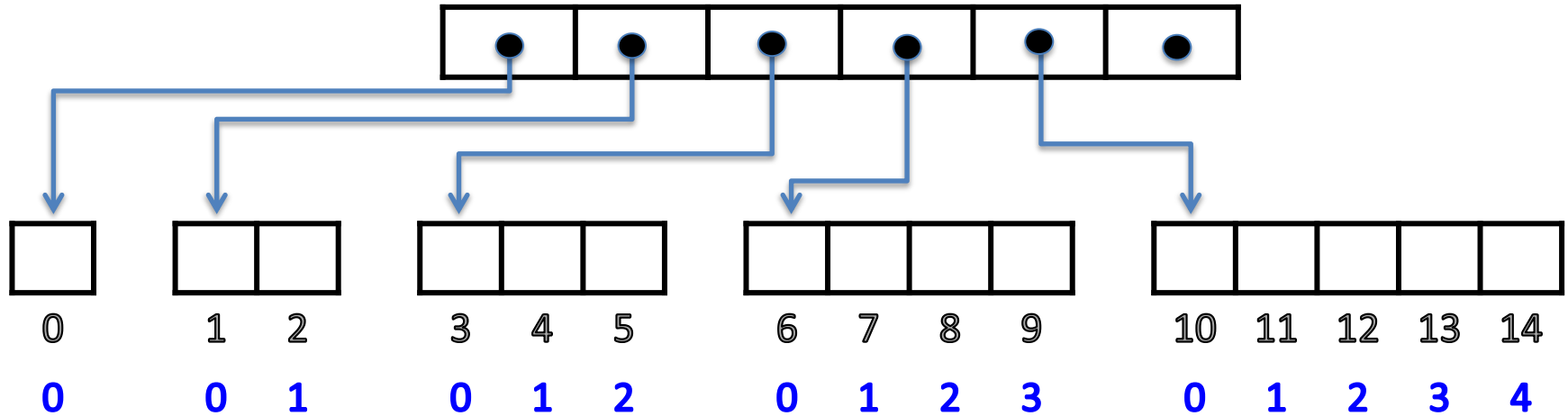
`shrink()` removes the last block whenever removing an element results in more than one empty block

RootishArrayStack



shrink() removes the last block whenever removing an elements results in more than one empty block

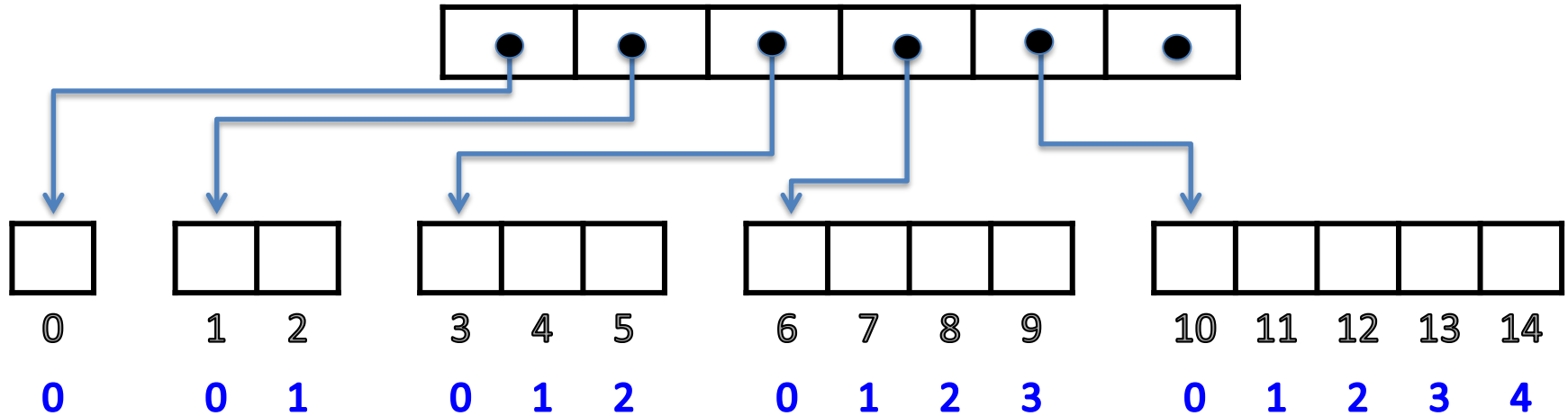
RootishArrayStack



What is cost of `grow()` and `shrink()`?

How often are they called?

RootishArrayStack



grow() add a new block to the end
shrink() removes the last block

What is cost?

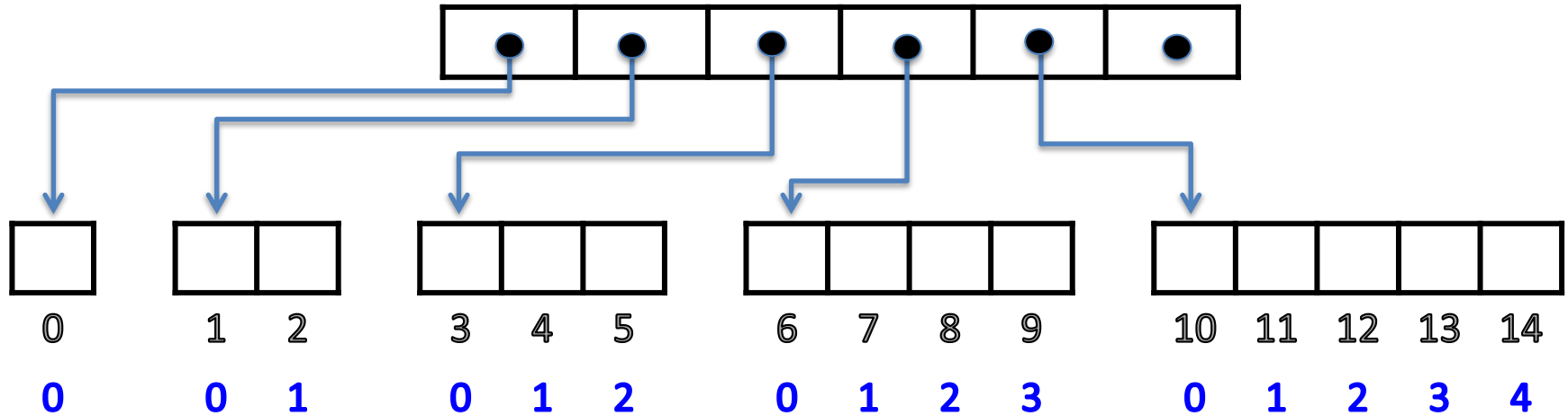
RootishArrayStack

Theorem 2.5: A RootishArrayStack implements the List interface. Ignoring the cost of `grow()` and `shrink()`, a RootishArrayStack supports the operations

- `get(i)/set(i,x)` in $O(1)$ time per operation, and
- `add(i,x)/remove(i)` in $O(1 + n - i)$ time per operation

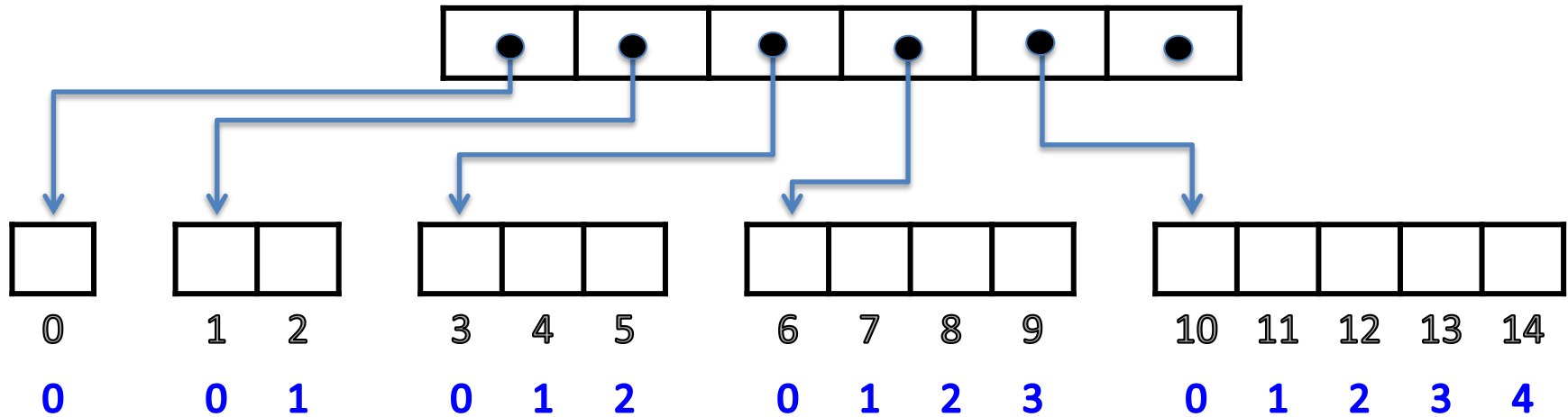
Both `grow()` and `shrink()` have amortized cost, over $m \geq 1$ add/remove operations, that is $O(m)/m$.

RootishArrayStack



How much space is wasted?

RootishArrayStack

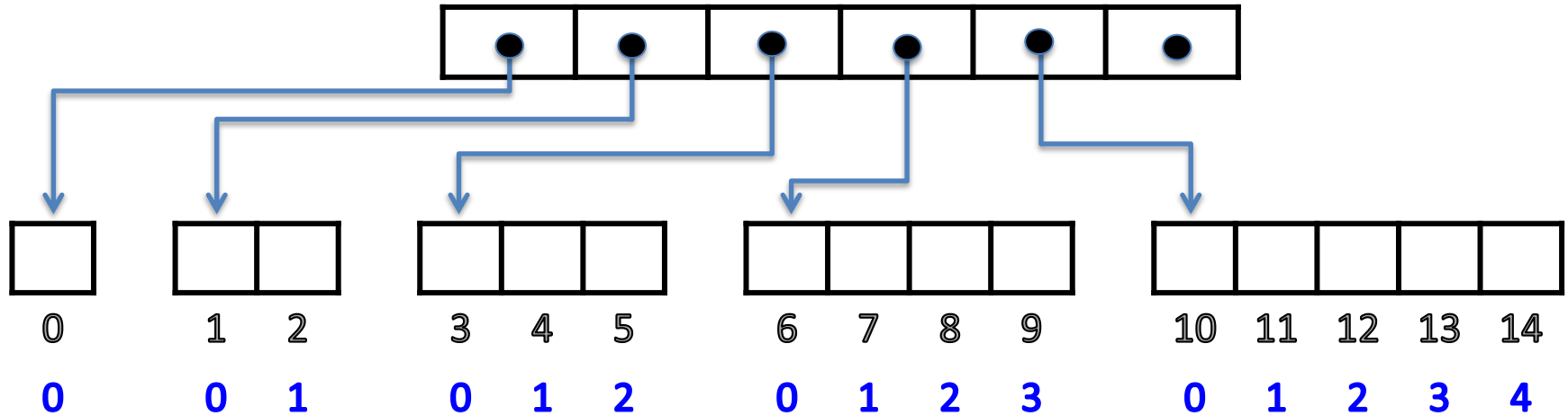


How much space is used (wasted)?

To store n elements we need r blocks such that

$$\frac{1}{2}r(r + 1) \geq n$$

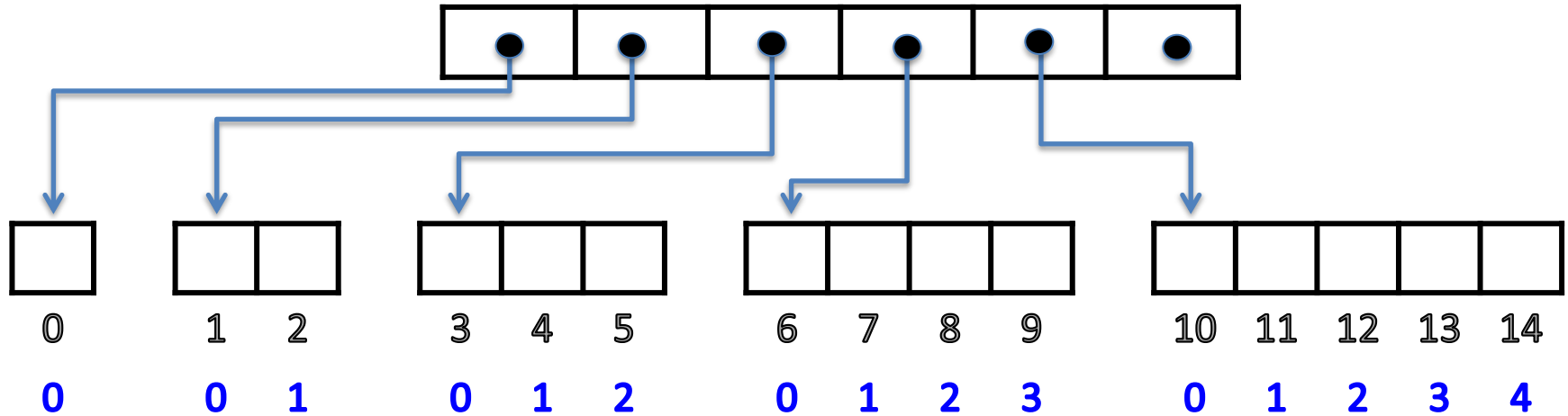
RootishArrayStack



We have at any given time

- n elements to store (using r blocks)
- A *blocks* list (size r)
- At most 2 empty blocks

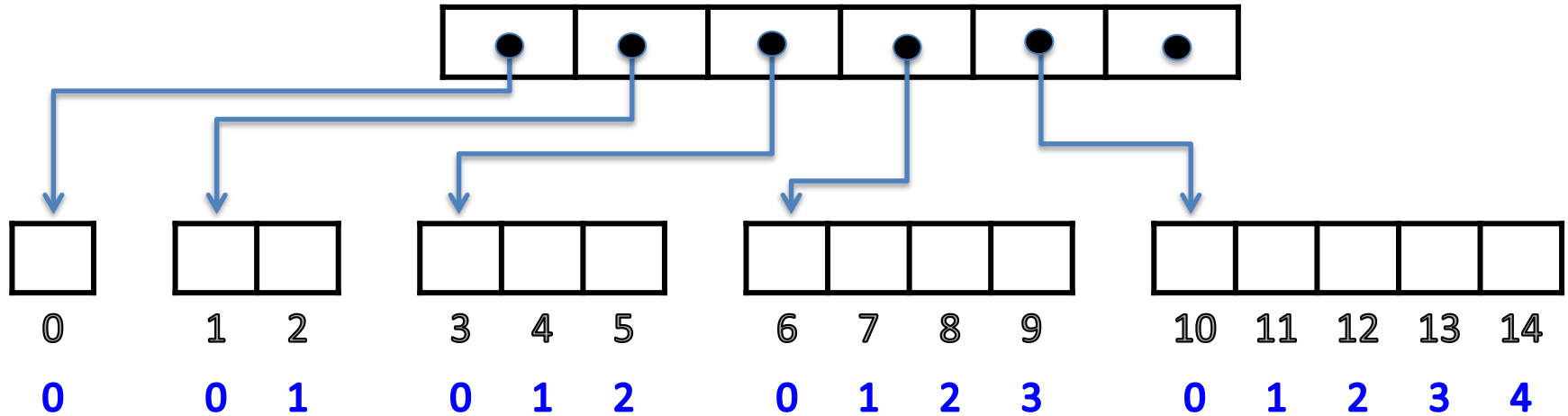
RootishArrayStack



We have at any given time

- n elements to store (using r blocks)
- A *blocks* list (size r)
- At most 2 empty blocks

RootishArrayStack



Total space is $n + O(\sqrt{n})$

RootishArrayStack

Theorem 2.5: A RootishArrayStack implements the List interface. Ignoring the cost of `grow()` and `shrink()`, a RootishArrayStack supports the operations

- `get(i)/set(i,x)` in $O(1)$ time per operation, and
- `add(i,x)/remove(i)` in $O(1 + n - i)$ time per operation

`grow()` and `shrink()` have amortized cost, over $m \geq 1$ `add/remove` operations, that is $O(1)$.

RootishArrayStack uses $n + O(\sqrt{n})$ space.

RootishArrayStack

Can we do any better?

No, $n + O(\sqrt{n})$ is optimal.

RootishArrayStack

Did we cheat?

Computing $\sqrt{n}$ is not a basic operation in the word-RAM model used (see §1.4)

See §2.6.4 for a discussion of why this is OK.

Coming Next...

Linked lists!

